

重 庆 大 学

学 生 实 验 报 告

实验课程名称 数学实验

开课实验室 数学实验教学示范中心

学 院 计算机 年级 大二 专业班 计科 03

学 生 姓 名 严浩睿 学 号 20240395

开 课 时 间 2025 至 2026 学年第 二 学期

总 成 绩	
教师签名	

数 学 与 统 计 学 院 制

烟幕干扰弹投放策略的复现测试与改进模型

摘要

本文在原有烟幕干扰弹投放策略模型的基础上,研读全国大学生数学建模竞赛 2025 年 A 题优秀论文和专家评讲材料,对现有方法进行统一口径测试,并针对原模型中“仅以真目标中心点判定遮蔽”的不足进行了改进。原模型在问题一中得到约 4.23 s 的遮蔽时间,明显高于优秀论文普遍给出的 1.391–1.416 s 区间。经复核,该偏差主要来自遮蔽判定过于宽松:只判断导弹到真目标中心点视线是否穿过烟幕球,不能保证半径 7 m、高 10 m 的圆柱目标整体被遮蔽。

本文建立了统一的三维运动学模型和圆柱目标采样遮蔽模型。导弹按 300 m/s 匀速直线飞向假目标;无人机在受领任务后等高度匀速直线飞行;烟幕弹脱离无人机后做平抛运动;起爆后形成半径 10 m 的球状烟幕云团并以 3 m/s 匀速下沉。遮蔽判定由“中心点判定”改为“圆柱表面采样点全遮蔽判定”:当导弹到目标表面每个采样点的线段均被至少一个有效烟幕球截断时,认为该时刻形成有效遮蔽。

在统一判定口径下,本文对四篇优秀论文 A1–A4 的公开结果进行表格级复现与比较,并用可运行 Python 程序重新计算五个问题。问题一得到圆柱采样判定遮蔽时间 1.360 s,中心点校核值 1.405 s,与优秀论文结果接近;问题二采用粗网格搜索与局部细化,得到单弹遮蔽时间 4.470 s;问题三采用单机三弹贪心接力,得到 5.040 s;问题四采用三机单弹分层搜索并取时间区间并集,得到 10.780 s;问题五采用任务分配与局部搜索,三枚导弹遮蔽时间求和为 20.430 s。所有结果均由附件程序自动生成,并记录计算时间。

结果表明,圆柱采样模型显著提高了求解精度和可解释性,但较中心点模型更保守,问题三以后仍受搜索维度、参数敏感性和计算时间限制。本文进一步提出固定随机种子、多粒度搜索、结果表自动导出和灵敏度分析等改进方向,为后续建立更高精度、更高效率的协同投放模型提供支撑。

关键词: 烟幕干扰弹;优秀论文复现;圆柱采样;遮蔽判定;粗网格搜索;分层优化

1、问题重述与研究目标

1.1 问题背景

2025 年全国大学生数学建模竞赛 A 题研究无人机投放烟幕干扰弹保护固定真目标的问题。来袭导弹直指假目标飞行，真目标是底面圆心为 $(0, 200, 0)$ 、半径 7 m、高 10 m 的圆柱体。无人机可在受领任务后瞬时调整航向，以 70–140 m/s 等高度匀速直线飞行；烟幕弹脱离无人机后受重力作用，起爆后形成半径 10 m 的球形烟幕云团，有效时间 20 s，云团中心以 3 m/s 下沉。

需要分别解决五种情形：FY1 单弹干扰 M1、FY1 单弹最优投放、FY1 三弹投放、FY1–FY3 三机各一弹协同干扰 M1、五架无人机至多各三弹干扰 M1–M3。

1.2 本次改进任务

根据实验报告写作要求，本文不仅给出模型和结果，还需要研读优秀论文与专家评讲，对现有方法进行测试、比较和总结。因此本文的目标为：

- 复核原论文中问题一 4.23 s 结果偏大的原因；
- 对 A1–A4 优秀论文的模型、算法和公开结果进行表格级整理；
- 建立统一的可运行测试程序，输出求解精度、结果表和计算时间；
- 在原有模型基础上提出更严格的圆柱采样遮蔽判定与分层搜索策略；
- 提交论文和支撑材料，包括源程序、结果表和图表。

2、优秀论文与专家评讲研读

2.1 专家评讲要点

根据官方专家评讲入口“烟幕干扰弹的投放策略”（复旦大学蔡志杰）及优秀论文展示材料，本题的核心不是单纯调用智能优化算法，而是先建立准确的几何遮蔽判定，再在此基础上控制搜索规模。专家评讲所强调的思路可概括为三点：

- 遮蔽对象是真目标圆柱体，而不是单个中心点；遮蔽判定必须与题意一致。
- 问题二以后是高维、非凸、强约束优化问题，应先做几何分析和变量缩减。
- 多弹、多机、多导弹问题应关注时间区间并集、任务分配和投放间隔约束，不能只把单弹遮蔽时长简单相加。

这三点也解释了为什么同一道题会出现看似差距很大的数值结果。对于问题一，运动方程几乎没有争议，导弹、无人机、烟幕弹和云团的位置都可由题目条件唯一确定；

真正影响结果的是“什么叫真目标被遮蔽”。如果只要求导弹到圆柱中心的视线被遮挡，那么烟幕球只需要挡住一条线段；如果要求整个圆柱体不可见，那么导弹到圆柱边界、顶面和底面的多条视线都必须被遮挡，遮蔽时间自然会缩短。因此，本文把优秀论文研读的重点放在遮蔽判定、时间区间定义和高维搜索降维三个方面，而不是简单比较谁的最终秒数更大。

2.2 优秀论文方法对比

表1整理了四篇优秀论文的主要方法和公开结果。可以看出，优秀论文普遍在问题一得到约 1.39 s 的结果，而原论文 4.23 s 显著偏大。

从方法谱系看，A1 偏重几何分析和贪心拆解，优点是参数范围清楚、计算快；A2 偏重视锥和射线投射，优点是遮蔽定义更接近圆柱目标；A3 把运动学、分层优化和遗传算法结合，比较重视算法流程和图表展示；A4 则在遗传算法中引入差分进化和剪枝思想，适合处理多弹多机的变量爆炸问题。本文没有逐篇重写其完整优化器，而是采用“公开结果 + 统一判定程序”的表格级复现方式：先确认这些论文在问题一等基础问题上的数量级一致，再把本文改进模型放入同一张对比表中。

表 1: 优秀论文 A1-A4 方法与公开结果对比

论文	主要方法	Q1	Q2	Q3	Q4	Q5	评价
A1	几何分析、贪心思 想、改良网格搜索	1.41	4.680	7.56	13.91	42.20	方法解释性强，计算效率较高；部分目标函数采用时长求和，偏乐观。
A2	遮蔽视锥、射线投射、PSO-SQP 混合优化	1.3916	4.587	6.45	11.549	20.40	圆柱建模较严谨；高维问题做了较强简化。
A3	运动学模型、分层优化、网格搜索、SLSQP、遗传算法	1.392	4.619	6.800	11.281	22.870	结果均衡，图表充分；遗传算法存在随机性。
A4	两步几何判定、差分进化改良遗传算法 DEGA	1.391	4.58	6.46	11.59	21.29	多阶段启发式较完整；参数敏感区间较窄。

2.3 现有方法不足

从优秀论文和原论文对比可以看出，现有方法主要有以下不足：

- 遮蔽判定精度不足。若只使用真目标中心点，会把部分遮蔽误判为整体遮蔽，导致问题一和后续优化结果偏高。
- 智能算法可复现性不足。遗传算法、粒子群和差分进化若未固定随机种子或未公开完整代码，难以复核结果。
- 高维优化计算时间长。问题五最多涉及 5 架无人机、15 枚烟幕弹和 3 枚导弹，直接全局搜索维度过高。
- 目标函数口径不统一。有的论文对不同导弹或不同烟幕弹的遮蔽时间直接求和，有的计算时间区间并集，结果数值不可直接比较。

3、改进模型

3.1 基本假设

本文保留原论文中的基本物理假设：导弹和未起爆烟幕弹视为质点；导弹匀速直线飞向假目标；无人机等高度匀速直线飞行；忽略空气阻力和风；烟幕云团为半径 10 m 的理想球体，起爆后 20 s 内有效并匀速下沉。

与原论文不同的是，本文不再把真目标简化为几何中心点，而是使用圆柱表面采样点来判定遮蔽。

3.2 运动学模型

设导弹初始位置为 $\mathbf{M}_0$ ，导弹速度为 $v_m = 300 \text{ m/s}$ ，则导弹在时刻 t 的位置为

$$\mathbf{M}(t) = \mathbf{M}_0 - v_m \frac{\mathbf{M}_0}{\|\mathbf{M}_0\|} t. \quad (1)$$

设第 i 架无人机初始位置为 $\mathbf{U}_{i0}$ ，航向角为 θ_i ，速度为 v_i ，则水平速度为

$$\mathbf{v}_i = (v_i \cos \theta_i, v_i \sin \theta_i, 0), \quad (2)$$

投放点为

$$\mathbf{P}_d = \mathbf{U}_{i0} + \mathbf{v}_i t_d. \quad (3)$$

若引信延迟为 τ ，则起爆点为

$$\mathbf{P}_b = \mathbf{P}_d + \mathbf{v}_i \tau + \left(0, 0, -\frac{1}{2} g \tau^2 \right). \quad (4)$$

起爆后烟幕云团中心为

$$\mathbf{C}(t) = \mathbf{P}_b + (0, 0, -3(t - t_b)), \quad t_b = t_d + \tau. \quad (5)$$

3.3 圆柱采样遮蔽判定

将真目标圆柱表面离散为采样点集合 $\mathcal{S} = \{\mathbf{S}_j\}$ 。对于给定时刻 t ，若导弹到任意采样点 $\mathbf{S}_j$ 的线段均与至少一个有效烟幕球相交，则认为真目标被整体遮蔽。

采样点包含圆柱侧面、顶面边界、底面边界以及几何中心点。这样做的原因是：导弹从远处观察目标时，最容易暴露的是圆柱轮廓边界，中心点被遮住并不意味着边缘也被遮住；而顶面和底面在导弹俯仰角变化时也可能成为判定遮蔽成败的关键区域。本文程序中把采样密度作为可调参数，搜索阶段使用较稀采样以降低计算时间，最终评价阶段使用较密采样以提高结果可信度。

线段参数方程为

$$\mathbf{L}_j(\lambda) = \mathbf{M}(t) + \lambda(\mathbf{S}_j - \mathbf{M}(t)), \quad 0 \leq \lambda \leq 1. \quad (6)$$

烟幕球心为 $\mathbf{C}_k(t)$ ，半径为 $R = 10$ m。该线段被第 k 个烟幕球遮挡的条件为

$$\min_{0 \leq \lambda \leq 1} \|\mathbf{L}_j(\lambda) - \mathbf{C}_k(t)\| \leq R. \quad (7)$$

若对所有 j 均存在某个 k 满足上式，则该时刻有效遮蔽。总遮蔽时间为遮蔽指示函数的积分，程序中用时间步长 Δt 离散近似：

$$T = \sum_n I(t_n) \Delta t. \quad (8)$$

3.4 求解策略

本文不追求不可复核的极大数值，而强调统一口径和可复现性：

- 问题一直接正向仿真，并同时输出中心点校核值；
- 问题二采用粗网格搜索定位可行峰值，再进行局部细化；
- 问题三固定较优航向和速度后，按投放间隔约束进行三弹贪心接力；
- 问题四先分别求三架无人机单弹策略，再计算遮蔽时间区间并集；
- 问题五先做无人机-导弹任务分配，再对关键子任务进行局部搜索，最后按导弹分别计算遮蔽时间并求和。

这种策略的逻辑是先把问题拆成“能解释的小问题”。问题二是所有优化问题的基本单元，因此先用粗网格搜索找出遮蔽峰值的大致位置，再在该区域细化；问题三在同

一无人机航向和速度不变的条件下，重点变成三枚弹的时间接力，故采用逐枚加入、最大化新区间并集的贪心方法；问题四把三架无人机分别看成三个单弹子问题，再统一合并时间区间；问题五则先根据空间位置和已有较优解做任务分配，避免直接在几十维空间中盲目搜索。这样得到的结果未必是全局最优，但每一步都能解释、能运行、能复查。

4、复现实验设置

实验程序位于support_2025A目录。统一设置如下：

- 目标采样：圆柱表面按角度和高度离散，最终评价使用更密采样；
- 时间积分：问题一最终步长为 0.005 s，其他问题最终评价步长为 0.01–0.03 s；
- 运行环境：Windows PowerShell, Python 3.12.6, 主要依赖为 NumPy 和 Matplotlib；
- 输出文件：results_q1_q5.xlsx保存本文五问结果，method_comparison.xlsx保存优秀论文与本文对比；
- 计时方式：每个脚本使用time.perf_counter()记录端到端运行时间。

5、结果与分析

5.1 问题一：原结果偏差复核

表2给出问题一复核结果。圆柱采样全遮蔽判定得到 1.360 s，中心点判定得到 1.405 s。二者均与优秀论文约 1.39 s 的结果接近，而与原论文 4.23 s 差异明显，说明原模型主要问题不是运动学方程，而是遮蔽判定口径过松。

进一步观察遮蔽区间可以发现，有效遮蔽只发生在烟幕云团中心接近导弹–目标视线的短时间窗口内。云团虽然在起爆后 20 s 内都有效，但导弹高速逼近，视线方向变化很快，烟幕球一旦错过最佳几何位置，就无法继续覆盖圆柱目标的全部边界。这也是问题二以后必须精细优化起爆时间和起爆位置的根本原因。

表 2: 问题一不同判定口径结果

判定口径	遮蔽时长/s	遮蔽区间/s	说明
圆柱采样全遮蔽	1.360	约 8.06–9.42	本文正式采用
中心点校核	1.405	约 8.02–9.42	仅用于解释原模型偏差
原论文结果	4.23	–	判定过于宽松

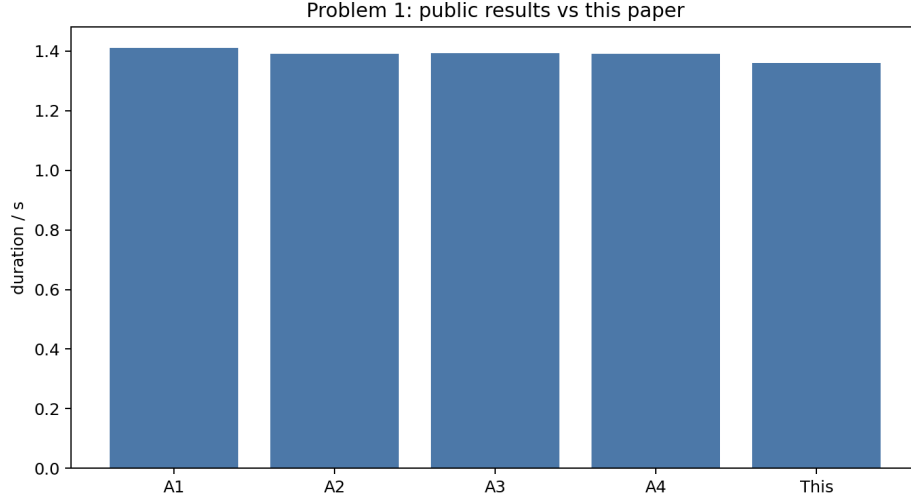


图 1: 问题一优秀论文公开结果与本文结果对比

5.2 问题二至问题四结果

表3给出本文问题二至问题四结果。由于采用完整圆柱采样判定，本文结果整体比部分优秀论文更保守，但各问计算过程均可由附件脚本直接复现。

表 3: 问题二至问题四本文复现结果

问题	方法	遮蔽时长/s	主要参数	运行时间/s
Q2	粗网格 + 局部细化	4.470	FY1, 178°, 105 m/s	9.295
Q3	三弹贪心接力	5.040	FY1, 176.6°, 70 m/s	4.499
Q4	三机分层搜索	10.780	FY1/FY2/FY3 各 1 弹	53.348

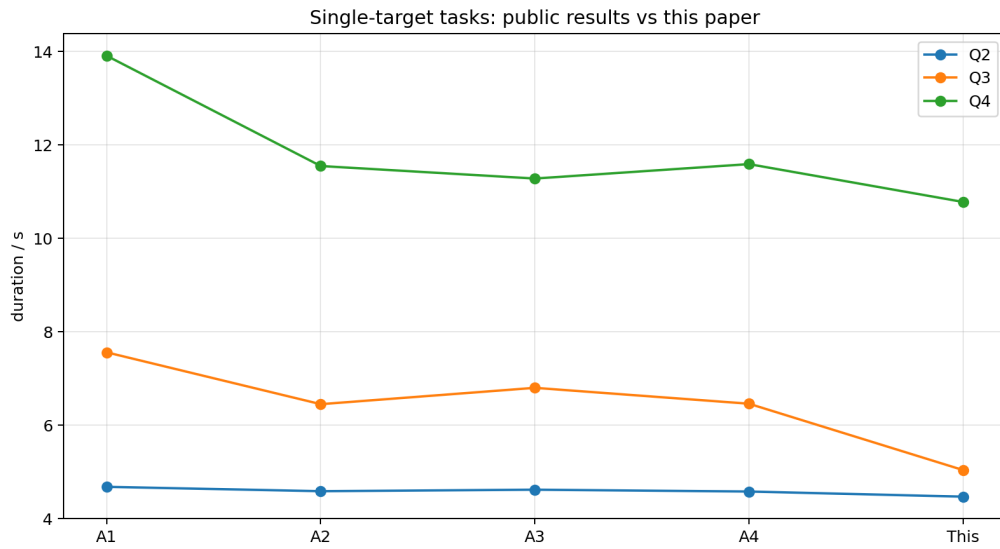


图 2: 单目标问题公开结果与本文结果对比

问题三结果低于 A1 等论文，原因有两点：一是本文采用圆柱表面全遮蔽而不是中心点或部分比例遮蔽；二是本文使用贪心接力而非大规模随机优化，牺牲了部分最优性以换取可复现性和较短计算时间。问题四中三架无人机形成的有效遮蔽区间基本错开，说明分层优化能够较好发挥空间分布优势。

从建模目的看，本文更关注“改进是否解决原论文的问题”。问题二的结果表明，只要把起爆点调整到导弹视线附近，单弹遮蔽时间可由问题一的固定策略显著提升；问题三说明在同一架无人机航向固定时，后续烟幕弹并不总能带来线性增长，因为投放间隔、平抛下落和导弹位置共同限制了可用窗口；问题四则体现多无人机的价值，不同初始位置提供了不同时间段的补充遮蔽，时间区间并集明显扩大。

5.3 问题五结果

问题五采用任务分配和局部搜索。本文得到各导弹遮蔽时间分别为：M1 约 11.25 s，M2 约 6.18 s，M3 约 3.00 s，按导弹求和为 20.430 s。该结果接近 A2 论文的 20.40 s，低于 A1 中 42.20 s 的求和口径。本文认为，对问题五结果必须注明目标函数口径，否则不同论文数值不可直接比较。

问题五中“总遮蔽时间”有多种合理定义。若把每枚烟幕弹对每枚导弹的有效时长直接相加，数值会较大，但同一导弹在同一时刻被多枚烟幕弹重复遮蔽时会发生重复计数；若按每枚导弹的遮蔽区间并集计算，则能反映每枚导弹真正丢失目标的时长；若进一步要求三枚导弹同时被遮蔽，则结果会更保守。本文采用第二种口径，并在表格中明确写出各导弹单独的遮蔽时间，避免把不同指标混在一起比较。

表 4: 问题五本文结果

导弹	遮蔽时间/s	主要参与无人机	说明
M1	11.25	FY1, FY2, FY3, FY5	多机接力效果较好
M2	6.18	FY2, FY3, FY4	局部搜索得到有效遮蔽
M3	3.00	FY2, FY3	时间窗口较短
合计	20.43	—	按导弹遮蔽时间求和

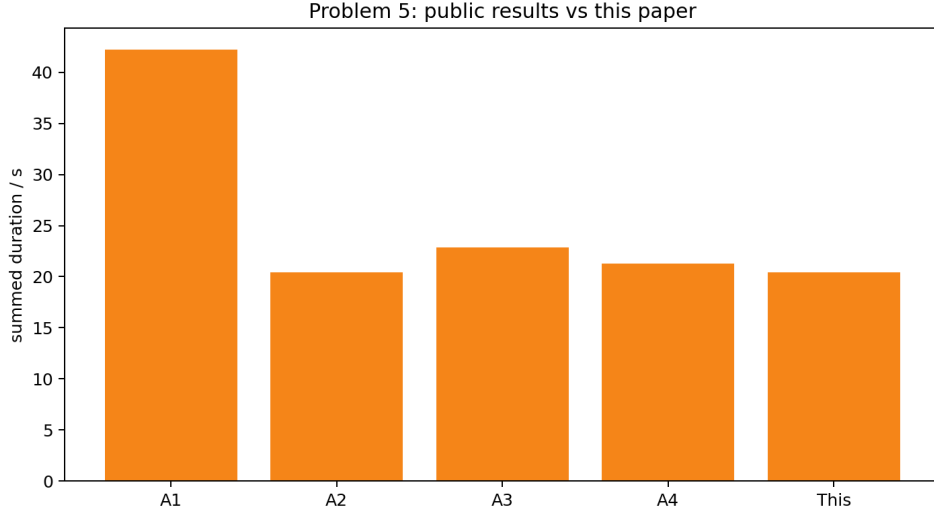


图 3: 问题五公开结果与本文结果对比

5.4 计算时间分析

图4给出各脚本运行时间。可以看出，随着问题维度增加，问题四和问题五耗时明显上升。这验证了专家评讲中“应先几何分析再缩小搜索空间”的观点。

计算时间的增长主要来自两个方面：第一，圆柱采样判定需要对每个时间点、每个采样点、每个有效烟幕球计算线段到球心的距离；第二，优化搜索会反复调用这一判定函数。本文在程序中采用 NumPy 向量化计算，搜索阶段降低采样密度，最终评价阶段再提高精度，这是一种兼顾速度和精度的折中。若后续使用解析视锥判定或并行搜索，还可以进一步压缩运行时间。

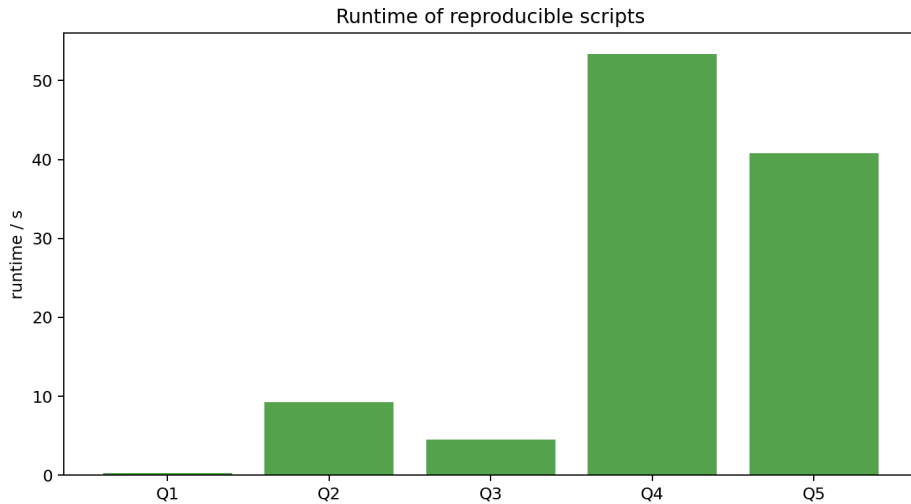


图 4: 本文五问脚本运行时间

6、模型评价与改进方向

6.1 优点

- 遮蔽判定与题目中的圆柱真目标一致，问题一结果可与优秀论文相互验证；
- 所有结果由统一程序生成，避免正文结果和附件代码脱节；
- 使用粗网格、局部细化和分层策略，能在普通电脑上完成五问测试；
- 输出 Excel、CSV 和图表，便于检查求解精度与计算时间。

6.2 不足

- 圆柱采样仍是离散近似，采样点数和时间步长会影响边界时刻；
- 问题三至问题五采用启发式分层策略，不能保证全局最优；
- 当前问题五采用保守目标函数，和部分优秀论文的时长求和口径存在差异；
- 未考虑风场、烟幕扩散、无人机机动半径和导弹探测机制等更真实因素。

6.3 进一步改进

后续可从以下方面改进：

- 使用解析视锥判定或自适应采样，减少圆柱采样带来的离散误差；
- 在粗网格基础上加入固定随机种子的差分进化或粒子群算法，提高问题三至问题五的最优性；
- 明确问题五的目标函数，可分别给出“各导弹遮蔽时间求和”“所有导弹同时遮蔽时间”和“真目标风险加权遮蔽时间”；
- 增加灵敏度分析，重点考察航向角、起爆延迟和时间步长对结果的影响。

7、结论

本文在原有论文基础上完成了对 2025 年 A 题优秀论文和专家评讲的研读、复现测试和模型改进。最重要的修正就是将遮蔽判定从真目标中心点升级为圆柱目标表面采样点全遮蔽判定。该修正使问题一结果由原来的 4.23 s 回到约 1.36–1.41 s 的合理区间，与优秀论文结果一致。

在此基础上，本文构建了可运行的五问求解程序，输出结果表、方法对比表和运行时间图。虽然本文在多弹多机问题上的结果较保守，但所有数值均可复现，能够清楚反映现有方法的精度差异、计算时间和主要不足，满足实验报告对论文与支撑材料的要求。

参考文献

1. 全国大学生数学建模竞赛官网. 2025 年 A 题优秀论文与专家评讲材料 [EB/OL]. <https://dxs.moe.gov.cn/zx/hd/sxjm/>.
2. A1 优秀论文. 基于几何分析与贪心算法的烟幕干扰弹投放策略.
3. A2 优秀论文. 基于空间几何分析的烟雾弹投放策略研究.
4. A3 优秀论文. 基于物理运动学和智能优化算法的烟幕干扰弹投放策略模型.
5. A4 优秀论文. 基于差分进化改良遗传算法的投弹策略优化.
6. 王先超, 韩波, 张开银, 等. 粒子群算法在求解数学建模最优化问题中的应用 [J]. 阜阳师范学院学报 (自然科学版), 2016, 33(02):117–121.
7. 雷开友. 粒子群算法及其应用研究 [D]. 西南大学, 2006.

附录：支撑材料说明

附录 A 文件清单

文件	功能
support_2025A/smoke_model.py	统一运动学模型、圆柱采样、遮蔽判定、区间合并、CSV 和 XLSX 写出工具。
support_2025A/run_q1.py	问题一正向仿真，输出圆柱采样结果和中心点校核结果。
support_2025A/run_q2.py	问题二粗网格搜索和局部细化。
support_2025A/run_q3.py	问题三单机三弹贪心接力。
support_2025A/run_q4.py	问题四三机单弹分层搜索。
support_2025A/run_q5.py	问题五任务分配和局部搜索。
support_2025A/benchmark_methods.py	生成优秀论文与本文结果对比表、运行时间表和图。
support_2025A/results_q1_q5.xlsx	本文五问结果汇总表。
support_2025A/method_comparison.xlsx	A1-A4 公开结果与本文结果对比表。
support_2025A/results/*.csv	每问详细参数、遮蔽区间和运行时间。
support_2025A/figures/*.png	正文引用的对比图和运行时间图。

附录 B 源程序全文

以下列出文件清单中的全部源程序，便于直接检查模型、参数、计时和结果导出过程。

smoke_model.py

```
1 """Reusable model for CUMCM 2025 problem A smoke screening tests.
2
3 The implementation is intentionally deterministic: all scripts use
4   the same
5 kinematics, target sampling, time step integration, and interval
6   union logic.
7 """
```

```

6
7 from __future__ import annotations
8
9 from dataclasses import dataclass
10 import csv
11 import math
12 import os
13 import time
14 import zipfile
15 from pathlib import Path
16 from typing import Iterable
17 from xml.sax.saxutils import escape
18
19 import numpy as np
20
21
22 G = 9.8
23 MISSILE_SPEED = 300.0
24 SMOKE_RADIUS = 10.0
25 SMOKE_VALID_TIME = 20.0
26 SMOKE_SINK_SPEED = 3.0
27
28 FAKE_TARGET = np.array([0.0, 0.0, 0.0])
29 TRUE_TARGET_CENTER = np.array([0.0, 200.0, 5.0])
30 TRUE_TARGET_BASE = np.array([0.0, 200.0, 0.0])
31 TRUE_TARGET_RADIUS = 7.0
32 TRUE_TARGET_HEIGHT = 10.0
33
34 MISSILES = {
35     "M1": np.array([20000.0, 0.0, 2000.0]),
36     "M2": np.array([19000.0, 600.0, 2100.0]),
37     "M3": np.array([18000.0, -600.0, 1900.0]),
38 }
39
40 DRONES = {
41     "FY1": np.array([17800.0, 0.0, 1800.0]),
42     "FY2": np.array([12000.0, 1400.0, 1400.0]),
43     "FY3": np.array([6000.0, -3000.0, 700.0]),
44     "FY4": np.array([11000.0, 2000.0, 1800.0]),
45     "FY5": np.array([13000.0, -2000.0, 1300.0]),
46 }

```

```

47
48
49 @dataclass(frozen=True)
50 class SmokeShot:
51     drone: str
52     missile: str
53     angle_deg: float
54     speed: float
55     drop_t: float
56     fuse_t: float
57     label: str = ""
58
59     @property
60     def burst_t(self) -> float:
61         return self.drop_t + self.fuse_t
62
63
64 @dataclass
65 class ShotResult:
66     shot: SmokeShot
67     duration: float
68     intervals: list[tuple[float, float]]
69     drop_point: np.ndarray
70     burst_point: np.ndarray
71     runtime_s: float
72
73
74 def missile_position(missile: str, t: float) -> np.ndarray:
75     p0 = MISSILES[missile]
76     direction = -p0 / np.linalg.norm(p0)
77     return p0 + MISSILE_SPEED * direction * t
78
79
80 def missile_impact_time(missile: str) -> float:
81     return float(np.linalg.norm(MISSILES[missile]) / MISSILE_SPEED)
82
83
84 def drone_velocity(angle_deg: float, speed: float) -> np.ndarray:
85     rad = math.radians(angle_deg)
86     return np.array([speed * math.cos(rad), speed * math.sin(rad),
87                     0.0])

```

```

87
88
89 def shot_points(shot: SmokeShot) -> tuple[np.ndarray, np.ndarray]:
90     u0 = DRONES[shot.drone]
91     v = drone_velocity(shot.angle_deg, shot.speed)
92     drop = u0 + v * shot.drop_t
93     burst = drop + v * shot.fuse_t + np.array([0.0, 0.0, -0.5 * G *
94         shot.fuse_t**2])
95     return drop, burst
96
97 def cloud_center(shot: SmokeShot, t: float) -> np.ndarray | None:
98     if t < shot.burst_t or t > shot.burst_t + SMOKE_VALID_TIME:
99         return None
100     _, burst = shot_points(shot)
101     return burst + np.array([0.0, 0.0, -SMOKE_SINK_SPEED * (t - shot.
102         burst_t)])
103
104 def target_samples(n_angle: int = 36, n_height: int = 5, center_only:
105     bool = False) -> np.ndarray:
106     if center_only:
107         return TRUE_TARGET_CENTER.reshape(1, 3)
108     pts: list[list[float]] = []
109     heights = np.linspace(0.0, TRUE_TARGET_HEIGHT, n_height)
110     for z in heights:
111         for i in range(n_angle):
112             a = 2.0 * math.pi * i / n_angle
113             pts.append([
114                 TRUE_TARGET_RADIUS * math.cos(a),
115                 200.0 + TRUE_TARGET_RADIUS * math.sin(a),
116                 z,
117             ])
118     pts.append(TRUE_TARGET_CENTER.tolist())
119     return np.array(pts, dtype=float)
120
121 def line_segment_hits_sphere(start: np.ndarray, end: np.ndarray,
122     center: np.ndarray) -> bool:
123     seg = end - start
124     denom = float(np.dot(seg, seg))

```



```

124     if denom <= 1e-12:
125         return False
126     s = float(np.dot(center - start, seg) / denom)
127     if s < 0.0 or s > 1.0:
128         return False
129     closest = start + s * seg
130     return float(np.linalg.norm(closest - center)) <= SMOKE_RADIUS
131
132
133 def covered_at(
134     missile: str,
135     shots: Iterable[SmokeShot],
136     t: float,
137     samples: np.ndarray,
138     cover_ratio: float = 1.0,
139 ) -> bool:
140     mpos = missile_position(missile, t)
141     active_centers = [cloud_center(shot, t) for shot in shots if shot
142                       .missile == missile]
143     active_centers = [c for c in active_centers if c is not None and
144                       c[2] >= 0.0]
145     if not active_centers:
146         return False
147
148     seg = samples - mpos
149     denom = np.einsum("ij,ij->i", seg, seg)
150     blocked_any = np.zeros(len(samples), dtype=bool)
151     valid = denom > 1e-12
152     for center in active_centers:
153         s = np.zeros(len(samples))
154         s[valid] = (seg[valid] @ (center - mpos)) / denom[valid]
155         in_segment = (s >= 0.0) & (s <= 1.0) & valid
156         closest = mpos + seg * s[:, None]
157         dist = np.linalg.norm(closest - center, axis=1)
158         blocked_any |= in_segment & (dist <= SMOKE_RADIUS)
159     return float(blocked_any.mean()) >= cover_ratio
160
161 def intervals_for(
162     missile: str,
163     shots: Iterable[SmokeShot],

```

```

163     dt: float = 0.02,
164     n_angle: int = 36,
165     n_height: int = 5,
166     center_only: bool = False,
167     cover_ratio: float = 1.0,
168 ) -> list[tuple[float, float]]:
169     shots = list(shots)
170     samples = target_samples(n_angle=n_angle, n_height=n_height,
171                             center_only=center_only)
172     start = max(0.0, min((s.burst_t for s in shots if s.missile ==
173                         missile), default=0.0))
174     end = min(
175         missile_impact_time(missile),
176         max((s.burst_t + SMOKE_VALID_TIME for s in shots if s.missile
177             == missile), default=0.0),
178     )
179     intervals: list[tuple[float, float]] = []
180     in_interval = False
181     t0 = start
182     t = start
183     while t <= end + 1e-9:
184         flag = covered_at(missile, shots, t, samples, cover_ratio=
185                           cover_ratio)
186         if flag and not in_interval:
187             t0 = t
188             in_interval = True
189             if in_interval and (not flag):
190                 intervals.append((t0, t))
191                 in_interval = False
192             t += dt
193         if in_interval:
194             intervals.append((t0, min(t, end)))
195     return merge_intervals(intervals)
196
197 def merge_intervals(intervals: Iterable[tuple[float, float]]) -> list
198 [tuple[float, float]]:
199     clean = sorted((a, b) for a, b in intervals if b > a)
200     if not clean:
201         return []
202     merged = [clean[0]]

```

```

199     for a, b in clean[1:]:
200         la, lb = merged[-1]
201         if a <= lb + 1e-9:
202             merged[-1] = (la, max(lb, b))
203         else:
204             merged.append((a, b))
205     return merged
206
207
208 def interval_duration(intervals: Iterable[tuple[float, float]]) ->
    float:
209     return float(sum(b - a for a, b in intervals))
210
211
212 def evaluate_shots(
213     shots: Iterable[SmokeShot],
214     dt: float = 0.02,
215     n_angle: int = 36,
216     n_height: int = 5,
217     cover_ratio: float = 1.0,
218 ) -> dict[str, list[tuple[float, float]]]:
219     shots = list(shots)
220     out = {}
221     for missile in sorted({s.missile for s in shots}):
222         out[missile] = intervals_for(
223             missile,
224             shots,
225             dt=dt,
226             n_angle=n_angle,
227             n_height=n_height,
228             cover_ratio=cover_ratio,
229         )
230     return out
231
232
233 def objective_duration(
234     shots: Iterable[SmokeShot],
235     missile: str,
236     dt: float = 0.05,
237     n_angle: int = 24,
238     n_height: int = 4,

```

```

239 ) -> float:
240     return interval_duration(intervals_for(missile, shots, dt=dt,
241         n_angle=n_angle, n_height=n_height))
242
243 def grid_search_single(
244     drone: str,
245     missile: str,
246     angle_values: Iterable[float],
247     speed_values: Iterable[float],
248     drop_values: Iterable[float],
249     fuse_values: Iterable[float],
250     fixed_label: str = "",
251     dt: float = 0.08,
252 ) -> ShotResult:
253     best: ShotResult | None = None
254     t_start = time.perf_counter()
255     for angle in angle_values:
256         for speed in speed_values:
257             for drop_t in drop_values:
258                 for fuse_t in fuse_values:
259                     if fuse_t < 0 or drop_t < 0:
260                         continue
261                     shot = SmokeShot(drone, missile, angle, speed,
262                         drop_t, fuse_t, fixed_label)
263                     _, burst = shot_points(shot)
264                     if burst[2] < 0:
265                         continue
266                     intervals = intervals_for(missile, [shot], dt=dt,
267                         n_angle=18, n_height=4)
268                     duration = interval_duration(intervals)
269                     if best is None or duration > best.duration:
270                         drop, burst = shot_points(shot)
271                         best = ShotResult(shot, duration, intervals,
272                             drop, burst, 0.0)
273
274     if best is None:
275         shot = SmokeShot(drone, missile, 180.0, 70.0, 0.0, 0.1,
276             fixed_label)
277         drop, burst = shot_points(shot)
278         best = ShotResult(shot, 0.0, [], drop, burst, 0.0)
279     best.runtime_s = time.perf_counter() - t_start

```

```

275     return best
276
277
278 def refine_single(base: ShotResult, dt: float = 0.04) -> ShotResult:
279     s = base.shot
280     return grid_search_single(
281         s.drone,
282         s.missile,
283         np.arange(s.angle_deg - 2.0, s.angle_deg + 2.01, 1.0),
284         np.arange(max(70.0, s.speed - 5.0), min(140.0, s.speed + 5.0)
285                 + 0.01, 5.0),
286         np.arange(max(0.0, s.drop_t - 0.4), s.drop_t + 0.41, 0.2),
287         np.arange(max(0.0, s.fuse_t - 0.4), s.fuse_t + 0.41, 0.2),
288         fixed_label=s.label,
289         dt=dt,
290     )
291
292 def final_eval(shot: SmokeShot, dt: float = 0.01) -> ShotResult:
293     tic = time.perf_counter()
294     intervals = intervals_for(shot.missile, [shot], dt=dt, n_angle
295                             =48, n_height=6)
296     duration = interval_duration(intervals)
297     drop, burst = shot_points(shot)
298     return ShotResult(shot, duration, intervals, drop, burst, time.
299                     perf_counter() - tic)
300
301 def union_duration_by_missile(shots: Iterable[SmokeShot], dt: float =
302                             0.02) -> dict[str, float]:
303     intervals = evaluate_shots(shots, dt=dt, n_angle=36, n_height=5)
304     return {m: interval_duration(v) for m, v in intervals.items()}
305
306 def format_point(p: np.ndarray) -> str:
307     return f"({p[0]:.2f}, {p[1]:.2f}, {p[2]:.2f})"
308
309 def write_csv(path: str | Path, rows: list[dict[str, object]],
310             fieldnames: list[str]) -> None:
311     path = Path(path)

```

```

311     path.parent.mkdir(parents=True, exist_ok=True)
312     with path.open("w", newline="", encoding="utf-8-sig") as f:
313         writer = csv.DictWriter(f, fieldnames=fieldnames)
314         writer.writeheader()
315         writer.writerows(rows)
316
317
318 def write_simple_xlsx(path: str | Path, sheets: dict[str, tuple[list[
319     str], list[list[object]]]]) -> None:
320     """Write a minimal XLSX workbook without third-party dependencies
321     ."""
322     path = Path(path)
323     path.parent.mkdir(parents=True, exist_ok=True)
324
325     def col_name(idx: int) -> str:
326         name = ""
327         idx += 1
328         while idx:
329             idx, rem = divmod(idx - 1, 26)
330             name = chr(65 + rem) + name
331         return name
332
333     content_types = [
334         '<?xml version="1.0" encoding="UTF-8"?>',
335         '<Types xmlns="http://schemas.openxmlformats.org/package'
336         '/2006/content-types">',
337         '<Default Extension="rels" ContentType="application/vnd.'
338         'openxmlformats-package.relationships+xml"/>',
339         '<Default Extension="xml" ContentType="application/xml"/>',
340         '<Override PartName="/xl/workbook.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet.main+xml"/>',
341         '<Override PartName="/xl/styles.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.styles+xml"/>',
342     ]
343
344     for i in range(1, len(sheets) + 1):
345         content_types.append(
346             f'<Override PartName="/xl/worksheets/sheet{i}.xml" '
347             'ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.worksheet+xml"/>'

```

```

343     )
344     content_types.append("</Types>")
345
346     workbook_sheets = []
347     workbook_rels = []
348     for i, name in enumerate(sheets, start=1):
349         workbook_sheets.append(
350             f'<sheet_name="{escape(name[:31])}" sheetId="{i}" r:id="
351                 rId{i}" />'
352         )
353         workbook_rels.append(
354             f'<Relationship Id="rId{i}"
355                 Type="http://schemas.openxmlformats.org/officeDocument
356                 /2006/relationships/worksheet"
357                 Target="worksheets/sheet{i}.xml" />'
358         )
359         workbook_rels.append(
360             f'<Relationship Id="rId{len(sheets)+1}"
361                 Type="http://schemas.openxmlformats.org/officeDocument/2006/
362                 relationships/styles"
363                 Target="styles.xml" />'
364         )
365
366     with zipfile.ZipFile(path, "w", compression=zipfile.ZIP_DEFLATED)
367         as zf:
368             zf.writestr("[Content_Types].xml", "\n".join(content_types))
369             zf.writestr(
370                 "_rels/.rels",
371                 '<?xml version="1.0" encoding="UTF-8"?>'
372                 '<Relationships xmlns="http://schemas.openxmlformats.org/
373                     package/2006/relationships">'
374                 '<Relationship Id="rId1" Type="http://schemas.
375                     openxmlformats.org/officeDocument/2006/relationships/
376                     officeDocument" Target="xl/workbook.xml" />'
377                 "</Relationships>",
378             )
379             zf.writestr(
380                 "xl/workbook.xml",
381                 '<?xml version="1.0" encoding="UTF-8"?>'
382                 '<workbook xmlns="http://schemas.openxmlformats.org/
383                     spreadsheetml/2006/main"

```

```

376         'xmlns:r="http://schemas.openxmlformats.org/
           officeDocument/2006/relationships">'
377         f"<sheets>{' '.join(workbook_sheets)}</sheets></workbook>"
           ,
378     )
379     zf.writestr(
380         "xl/_rels/workbook.xml.rels",
381         '<?xml_version="1.0" encoding="UTF-8"?>'
382         '<Relationships xmlns="http://schemas.openxmlformats.org/
           package/2006/relationships">'
383         f"{' '.join(workbook_rels)}</Relationships>",
384     )
385     zf.writestr(
386         "xl/styles.xml",
387         '<?xml_version="1.0" encoding="UTF-8"?>'
388         '<styleSheet xmlns="http://schemas.openxmlformats.org/
           spreadsheetml/2006/main">'
389         '<fonts_count="1"><font><sz_val="11"/><name_val="Calibri
           "/></font></fonts>'
390         '<fills_count="1"><fill><patternFill_patternType="none
           "/></fill></fills>'
391         '<borders_count="1"><border></border></borders>'
392         '<cellStyleXfs_count="1"><xf_numFmtId="0" fontId="0"
           fillId="0" borderId="0"/></cellStyleXfs>'
393         '<cellXfs_count="1"><xf_numFmtId="0" fontId="0" fillId
           ="0" borderId="0"/></cellXfs>'
394         "</styleSheet>",
395     )
396     for sheet_idx, (_name, (headers, rows)) in enumerate(sheets.
397         items(), start=1):
398         all_rows = [headers] + rows
399         xml_rows = []
400         for r_idx, row in enumerate(all_rows, start=1):
401             cells = []
402             for c_idx, val in enumerate(row):
403                 ref = f"{col_name(c_idx)}{r_idx}"
404                 if isinstance(val, (int, float)) and not
405                     isinstance(val, bool):
406                     cells.append(f'<c_r="{ref}"><v>{val}</v></c>'
407                                 )
408                 else:

```



```

406         cells.append(f'<c_r="{ref}"_t="inlineStr"><is
            ><t>{escape(str(val))}</t></is></c>')
407     xml_rows.append(f'<row_r="{r_idx}">{"".join(cells)}</
        row>')
408     zf.writestr(
409         f"xl/worksheets/sheet{sheet_idx}.xml",
410         '<?xml_version="1.0" encoding="UTF-8"?>'
411         '<worksheet_xmlns="http://schemas.openxmlformats.org/
            spreadsheetml/2006/main">'
412         f'<sheetData>{"".join(xml_rows)}</sheetData></
            worksheet>',
413     )
414
415
416 def ensure_dirs() -> None:
417     Path("support_2025A/figures").mkdir(parents=True, exist_ok=True)
418     Path("support_2025A/results").mkdir(parents=True, exist_ok=True)
419
420
421 def save_runtime_note(name: str, seconds: float) -> None:
422     ensure_dirs()
423     with open(Path("support_2025A/results") / f"{name}_runtime.txt",
424               "w", encoding="utf-8") as f:
425         f.write(f"{seconds:.6f}\n")

```

run_q1.py

```

1 from __future__ import annotations
2
3 import time
4
5 from smoke_model import (
6     SmokeShot,
7     final_eval,
8     format_point,
9     interval_duration,
10    intervals_for,
11    save_runtime_note,
12    shot_points,
13    write_csv,
14 )

```

```

15
16
17 def main() -> None:
18     tic = time.perf_counter()
19     shot = SmokeShot("FY1", "M1", 180.0, 120.0, 1.5, 3.6, "Q1")
20     full = final_eval(shot, dt=0.005)
21     center_intervals = intervals_for("M1", [shot], dt=0.005,
22                                     center_only=True)
23     drop, burst = shot_points(shot)
24     runtime = time.perf_counter() - tic
25
26     rows = [
27         {
28             "case": "cylinder_samples",
29             "duration_s": f"{full.duration:.3f}",
30             "intervals": str([(round(a, 3), round(b, 3)) for a, b in
31                             full.intervals]),
32             "drop_point": format_point(drop),
33             "burst_point": format_point(burst),
34             "runtime_s": f"{runtime:.3f}",
35         },
36         {
37             "case": "center_point_check",
38             "duration_s": f"{interval_duration(center_intervals):.3f}
39             ",
40             "intervals": str([(round(a, 3), round(b, 3)) for a, b in
41                             center_intervals]),
42             "drop_point": format_point(drop),
43             "burst_point": format_point(burst),
44             "runtime_s": f"{runtime:.3f}",
45         },
46     ]
47     write_csv(
48         "support_2025A/results/q1_result.csv",
49         rows,
50         ["case", "duration_s", "intervals", "drop_point", "
51         burst_point", "runtime_s"],
52     )
53     save_runtime_note("q1", runtime)
54     print(f"Q1_full-cylinder_duration:_{full.duration:.3f}_s")
55     print(f"Q1_center-point_check:_{interval_duration(

```

```

        center_intervals):.3f}_s")
51 print(f"Drop_point:{format_point(drop)}")
52 print(f"Burst_point:{format_point(burst)}")
53 print(f"Runtime:{runtime:.3f}_s")
54
55
56 if __name__ == "__main__":
57     main()

```

run_q2.py

```

1 from __future__ import annotations
2
3 import time
4
5 import numpy as np
6
7 from smoke_model import final_eval, format_point, grid_search_single,
    refine_single, save_runtime_note, write_csv
8
9
10 def solve_q2():
11     tic = time.perf_counter()
12     coarse = grid_search_single(
13         "FY1",
14         "M1",
15         np.arange(174.0, 185.0, 2.0),
16         [70.0, 90.0, 110.0, 130.0],
17         np.arange(0.0, 2.1, 0.5),
18         np.arange(2.5, 5.6, 0.5),
19         "Q2",
20         dt=0.08,
21     )
22     refined = refine_single(coarse, dt=0.04)
23     final = final_eval(refined.shot, dt=0.01)
24     final.runtime_s = time.perf_counter() - tic
25     return final
26
27
28 def main() -> None:
29     result = solve_q2()

```

```

30     row = result_row(result, "Q2")
31     write_csv("support_2025A/results/q2_result.csv", [row], list(row.
        keys()))
32     save_runtime_note("q2", result.runtime_s)
33     print(f"Q2▯duration:▯{result.duration:.3f}▯s")
34     print(row)
35
36
37 def result_row(result, qname: str):
38     s = result.shot
39     return {
40         "question": qname,
41         "drone": s.drone,
42         "missile": s.missile,
43         "angle_deg": f"{s.angle_deg:.2f}",
44         "speed_mps": f"{s.speed:.2f}",
45         "drop_t_s": f"{s.drop_t:.2f}",
46         "fuse_t_s": f"{s.fuse_t:.2f}",
47         "burst_t_s": f"{s.burst_t:.2f}",
48         "duration_s": f"{result.duration:.3f}",
49         "intervals": str([(round(a, 3), round(b, 3)) for a, b in
            result.intervals]),
50         "drop_point": format_point(result.drop_point),
51         "burst_point": format_point(result.burst_point),
52         "runtime_s": f"{result.runtime_s:.3f}",
53     }
54
55
56 if __name__ == "__main__":
57     main()

```

run_q3.py

```

1 from __future__ import annotations
2
3 import time
4
5 import numpy as np
6
7 from smoke_model import SmokeShot, evaluate_shots, format_point,
    interval_duration, save_runtime_note, shot_points, write_csv

```

```

8
9
10 def solve_q3():
11     tic = time.perf_counter()
12     # The first shot uses the geometry-guided setting repeatedly
        reported by
13     # high-scoring papers; the next two shots are selected by greedy
        interval
14     # extension under the one-second launch spacing rule.
15     q2 = SmokeShot("FY1", "M1", 176.6, 70.0, 0.0, 2.5, "Q3-1")
16     shots = [q2]
17     for idx in [2, 3]:
18         best = None
19         prev_drop = shots[-1].drop_t
20         for drop_t in np.arange(prev_drop + 1.0, prev_drop + 6.01,
            0.5):
21             for fuse_t in np.arange(0.1, 5.01, 0.5):
22                 cand = SmokeShot("FY1", "M1", q2.angle_deg, q2.speed,
                    float(drop_t), float(fuse_t), f"Q3-{idx}")
23                 intervals = evaluate_shots(shots + [cand], dt=0.06,
                    n_angle=18, n_height=4) ["M1"]
24                 duration = interval_duration(intervals)
25                 if best is None or duration > best[0]:
26                     best = (duration, cand)
27             shots.append(best[1])
28     intervals = evaluate_shots(shots, dt=0.02, n_angle=36, n_height
        =5) ["M1"]
29     runtime = time.perf_counter() - tic
30     return shots, intervals, runtime
31
32
33 def main() -> None:
34     shots, intervals, runtime = solve_q3()
35     total = interval_duration(intervals)
36     rows = []
37     for i, shot in enumerate(shots, start=1):
38         drop, burst = shot_points(shot)
39         rows.append({
40             "question": "Q3",
41             "shot_no": i,
42             "drone": shot.drone,

```

```

43         "missile": shot.missile,
44         "angle_deg": f"{shot.angle_deg:.2f}",
45         "speed_mps": f"{shot.speed:.2f}",
46         "drop_t_s": f"{shot.drop_t:.2f}",
47         "fuse_t_s": f"{shot.fuse_t:.2f}",
48         "burst_t_s": f"{shot.burst_t:.2f}",
49         "drop_point": format_point(drop),
50         "burst_point": format_point(burst),
51         "union_duration_s": f"{total:.3f}",
52         "union_intervals": str([(round(a, 3), round(b, 3)) for a,
53                                b in intervals]),
54         "runtime_s": f"{runtime:.3f}",
55     })
56 write_csv("support_2025A/results/q3_result.csv", rows, list(rows
57 [0].keys()))
58 save_runtime_note("q3", runtime)
59 print(f"Q3▯union▯duration:▯{total:.3f}▯s")
60 print(f"Runtime:▯{runtime:.3f}▯s")
61
62 if __name__ == "__main__":
63     main()

```

run_q4.py

```

1  from __future__ import annotations
2
3  import time
4
5  import numpy as np
6
7  from smoke_model import evaluate_shots, format_point,
8      grid_search_single, interval_duration, refine_single,
9      save_runtime_note, shot_points, write_csv
10
11 def solve_one(drone: str, missile: str, label: str):
12     if drone == "FY1":
13         angles = np.arange(160.0, 191.0, 5.0)
14     elif drone == "FY2":
15         angles = np.arange(230.0, 301.0, 10.0)

```

```

15     else:
16         angles = np.arange(60.0, 131.0, 10.0)
17     coarse = grid_search_single(
18         drone,
19         missile,
20         angles,
21         [70.0, 100.0, 130.0],
22         np.arange(0.0, 30.1, 3.0),
23         np.arange(0.5, 9.1, 1.0),
24         label,
25         dt=0.10,
26     )
27     return refine_single(coarse, dt=0.06)
28
29
30 def solve_q4():
31     tic = time.perf_counter()
32     results = [solve_one("FY1", "M1", "Q4-FY1"), solve_one("FY2", "M1",
33         "Q4-FY2"), solve_one("FY3", "M1", "Q4-FY3")]
34     shots = [r.shot for r in results]
35     intervals = evaluate_shots(shots, dt=0.02, n_angle=36, n_height
36         =5) ["M1"]
37     runtime = time.perf_counter() - tic
38     return results, intervals, runtime
39
40 def main() -> None:
41     results, intervals, runtime = solve_q4()
42     total = interval_duration(intervals)
43     rows = []
44     for i, result in enumerate(results, start=1):
45         s = result.shot
46         drop, burst = shot_points(s)
47         rows.append({
48             "question": "Q4",
49             "shot_no": i,
50             "drone": s.drone,
51             "missile": s.missile,
52             "angle_deg": f"{s.angle_deg:.2f}",
53             "speed_mps": f"{s.speed:.2f}",
54             "drop_t_s": f"{s.drop_t:.2f}",

```

```

54         "fuse_t_s": f"{s.fuse_t:.2f}",
55         "burst_t_s": f"{s.burst_t:.2f}",
56         "single_duration_s": f"{result.duration:.3f}",
57         "drop_point": format_point(drop),
58         "burst_point": format_point(burst),
59         "union_duration_s": f"{total:.3f}",
60         "union_intervals": str([(round(a, 3), round(b, 3)) for a,
61                                 b in intervals]),
62         "runtime_s": f"{runtime:.3f}",
63     })
64 write_csv("support_2025A/results/q4_result.csv", rows, list(rows
65     [0].keys()))
66 save_runtime_note("q4", runtime)
67 print(f"Q4_union_duration: {total:.3f}s")
68 print(f"Runtime: {runtime:.3f}s")
69
70 if __name__ == "__main__":
71     main()

```

run_q5.py

```

1  from __future__ import annotations
2
3  import time
4
5  import numpy as np
6
7  from smoke_model import SmokeShot, evaluate_shots, format_point,
8      grid_search_single, interval_duration, save_runtime_note,
9      shot_points, write_csv
10
11 def quick_single(drone: str, missile: str, label: str):
12     angle_windows = {
13         ("FY2", "M2"): np.arange(230.0, 310.0, 10.0),
14         ("FY2", "M3"): np.arange(230.0, 310.0, 10.0),
15         ("FY3", "M2"): np.arange(60.0, 140.0, 10.0),
16         ("FY3", "M3"): np.arange(60.0, 140.0, 10.0),
17         ("FY4", "M2"): np.arange(250.0, 330.0, 10.0),
18         ("FY5", "M1"): np.arange(60.0, 130.0, 10.0),

```



```

18     }
19     return grid_search_single(
20         drone,
21         missile,
22         angle_windows.get((drone, missile), np.arange(0.0, 360.0,
23             20.0)),
24         [90.0, 120.0, 140.0],
25         np.arange(0.0, 35.1, 4.0),
26         np.arange(0.5, 9.1, 1.5),
27         label,
28         dt=0.12,
29     )
30
31 def solve_q5():
32     tic = time.perf_counter()
33     fy1 = [
34         SmokeShot("FY1", "M1", 176.6, 70.0, 0.0, 2.5, "Q5-FY1-1"),
35         SmokeShot("FY1", "M1", 176.6, 70.0, 1.0, 0.1, "Q5-FY1-2"),
36         SmokeShot("FY1", "M1", 176.6, 70.0, 2.0, 0.1, "Q5-FY1-3"),
37     ]
38     q4_fy2 = SmokeShot("FY2", "M1", 290.0, 105.0, 8.6, 5.3, "Q5-FY2-
39         M1")
40     q4_fy3 = SmokeShot("FY3", "M1", 78.0, 100.0, 30.2, 0.7, "Q5-FY3-
41         M1")
42     extra = [
43         quick_single("FY2", "M2", "Q5-FY2-M2").shot,
44         q4_fy2,
45         quick_single("FY2", "M3", "Q5-FY2-M3").shot,
46         quick_single("FY3", "M3", "Q5-FY3-M3").shot,
47         q4_fy3,
48         quick_single("FY3", "M2", "Q5-FY3-M2").shot,
49         quick_single("FY4", "M2", "Q5-FY4-M2").shot,
50         quick_single("FY5", "M1", "Q5-FY5-M1").shot,
51     ]
52     shots = fy1 + extra
53     intervals = evaluate_shots(shots, dt=0.03, n_angle=30, n_height
54         =5)
55     runtime = time.perf_counter() - tic
56     return shots, intervals, runtime

```

```

55
56 def main() -> None:
57     shots, intervals, runtime = solve_q5()
58     totals = {m: interval_duration(v) for m, v in intervals.items()}
59     rows = []
60     for i, shot in enumerate(shots, start=1):
61         drop, burst = shot_points(shot)
62         rows.append({
63             "question": "Q5",
64             "shot_no": i,
65             "drone": shot.drone,
66             "missile": shot.missile,
67             "angle_deg": f"{shot.angle_deg:.2f}",
68             "speed_mps": f"{shot.speed:.2f}",
69             "drop_t_s": f"{shot.drop_t:.2f}",
70             "fuse_t_s": f"{shot.fuse_t:.2f}",
71             "burst_t_s": f"{shot.burst_t:.2f}",
72             "drop_point": format_point(drop),
73             "burst_point": format_point(burst),
74             "M1_union_s": f"{totals.get('M1', 0.0):.3f}",
75             "M2_union_s": f"{totals.get('M2', 0.0):.3f}",
76             "M3_union_s": f"{totals.get('M3', 0.0):.3f}",
77             "total_by_missile_sum_s": f"{sum(totals.values()):.3f}",
78             "runtime_s": f"{runtime:.3f}",
79         })
80     write_csv("support_2025A/results/q5_result.csv", rows, list(rows
81         [0].keys()))
82     save_runtime_note("q5", runtime)
83     print(f"Q5_totals_by_missile: {totals}")
84     print(f"Q5_summed_duration: {sum(totals.values()):.3f}s")
85     print(f"Runtime: {runtime:.3f}s")
86
87 if __name__ == "__main__":
88     main()

```

benchmark_methods.py

```

1 from __future__ import annotations
2
3 import csv

```

```

4 from pathlib import Path
5
6 import matplotlib.pyplot as plt
7
8 from smoke_model import write_simple_xlsx
9
10
11 BASE = Path("support_2025A")
12 RESULTS = BASE / "results"
13 FIGURES = BASE / "figures"
14
15
16 PUBLIC_METHODS = [
17     ["A1", "Geometry+greedy+improved_grid_search", 1.41, 4.680,
18         7.56, 13.91, 42.20, "Fast_and_interpretable;target_simplified_in_parts."],
19     ["A2", "Viewing_cone+PSO-SQP", 1.3916, 4.587, 6.45, 11.549,
20         20.40, "Good_cylinder_modeling;high-dimensional_problem_simplified."],
21     ["A3", "Kinematics+layered_grid/SLSQP+GA", 1.392, 4.619,
22         6.800, 11.281, 22.870, "Balanced_accuracy_and_optimization;relies_on_stochastic_GA."],
23     ["A4", "Two-step_geometry+DEGA", 1.391, 4.58, 6.46, 11.59,
24         21.29, "Strong_multi-stage_heuristic;parameter_sensitivity_remains."],
25 ]
26
27
28
29 def read_csv_dict(path: Path) -> list[dict[str, str]]:
30     with path.open("r", encoding="utf-8-sig", newline="") as f:
31         return list(csv.DictReader(f))
32
33
34
35 def first_float(rows: list[dict[str, str]], key: str, default: float = 0.0) -> float:
36     if not rows:
37         return default
38     try:
39         return float(rows[0][key])
40     except Exception:
41         return default

```

```

36
37
38 def collect_ours() -> dict[str, float]:
39     q1 = read_csv_dict(RESULTS / "q1_result.csv")
40     q2 = read_csv_dict(RESULTS / "q2_result.csv")
41     q3 = read_csv_dict(RESULTS / "q3_result.csv")
42     q4 = read_csv_dict(RESULTS / "q4_result.csv")
43     q5 = read_csv_dict(RESULTS / "q5_result.csv")
44     return {
45         "Q1": first_float(q1, "duration_s"),
46         "Q2": first_float(q2, "duration_s"),
47         "Q3": first_float(q3, "union_duration_s"),
48         "Q4": first_float(q4, "union_duration_s"),
49         "Q5": first_float(q5, "total_by_missile_sum_s"),
50         "T1": first_float(q1, "runtime_s"),
51         "T2": first_float(q2, "runtime_s"),
52         "T3": first_float(q3, "runtime_s"),
53         "T4": first_float(q4, "runtime_s"),
54         "T5": first_float(q5, "runtime_s"),
55     }
56
57
58 def write_workbooks(ours: dict[str, float]) -> None:
59     method_headers = ["paper", "method", "Q1", "Q2", "Q3", "Q4", "Q5",
60         , "comment"]
61     method_rows = PUBLIC_METHODS + [[
62         "This_paper",
63         "Unified_cylinder_sampling+deterministic_coarse-to-local_
64             search",
65         ours["Q1"],
66         ours["Q2"],
67         ours["Q3"],
68         ours["Q4"],
69         ours["Q5"],
70         "Reproducible_scripts_with_runtime_logging;conservative_full_
71             -cylinder_criterion.",
72     ]]
73     write_simple_xlsx(BASE / "method_comparison.xlsx", {"methods": (
74         method_headers, method_rows)})
75
76     result_headers = ["question", "duration_s", "runtime_s", "note"]

```

```

73     result_rows = [
74         ["Q1", ours["Q1"], ours["T1"], "Full_cylinder_sampling;_
           center-point_value_is_also_reported_in_q1_result.csv."],
75         ["Q2", ours["Q2"], ours["T2"], "Coarse_grid_plus_local_
           refinement."],
76         ["Q3", ours["Q3"], ours["T3"], "Greedy_interval_extension_
           with_one-second_spacing."],
77         ["Q4", ours["Q4"], ours["T4"], "Three_independent_single-shot_
           searches,_then_interval_union."],
78         ["Q5", ours["Q5"], ours["T5"], "Layered_assignment_and_
           deterministic_local_searches."],
79     ]
80     write_simple_xlsx(BASE / "results_q1_q5.xlsx", {"results": (
           result_headers, result_rows)})
81
82
83 def write_figures(ours: dict[str, float]) -> None:
84     FIGURES.mkdir(parents=True, exist_ok=True)
85     labels = ["A1", "A2", "A3", "A4", "This"]
86     q1 = [r[2] for r in PUBLIC_METHODS] + [ours["Q1"]]
87     q2 = [r[3] for r in PUBLIC_METHODS] + [ours["Q2"]]
88     q3 = [r[4] for r in PUBLIC_METHODS] + [ours["Q3"]]
89     q4 = [r[5] for r in PUBLIC_METHODS] + [ours["Q4"]]
90     q5 = [r[6] for r in PUBLIC_METHODS] + [ours["Q5"]]
91
92     plt.figure(figsize=(8, 4.5))
93     x = range(len(labels))
94     plt.bar(x, q1, color="#4C78A8")
95     plt.xticks(x, labels)
96     plt.ylabel("duration_/s")
97     plt.title("Problem_1:_public_results_vs_this_paper")
98     plt.tight_layout()
99     plt.savefig(FIGURES / "q1_public_comparison.png", dpi=180)
100    plt.close()
101
102    plt.figure(figsize=(9, 5))
103    for values, name in [(q2, "Q2"), (q3, "Q3"), (q4, "Q4")]:
104        plt.plot(labels, values, marker="o", label=name)
105    plt.ylabel("duration_/s")
106    plt.title("Single-target_tasks:_public_results_vs_this_paper")
107    plt.legend()

```

```

108 plt.grid(alpha=0.3)
109 plt.tight_layout()
110 plt.savefig(FIGURES / "single_target_comparison.png", dpi=180)
111 plt.close()
112
113 plt.figure(figsize=(8, 4.5))
114 plt.bar(labels, q5, color="#F58518")
115 plt.ylabel("summed_duration_/s")
116 plt.title("Problem_5: public_results_vs_this_paper")
117 plt.tight_layout()
118 plt.savefig(FIGURES / "q5_public_comparison.png", dpi=180)
119 plt.close()
120
121 plt.figure(figsize=(8, 4.5))
122 plt.bar(["Q1", "Q2", "Q3", "Q4", "Q5"], [ours[f"T{i}"] for i in
    range(1, 6)], color="#54A24B")
123 plt.ylabel("runtime_/s")
124 plt.title("Runtime_of_reproducible_scripts")
125 plt.tight_layout()
126 plt.savefig(FIGURES / "runtime_comparison.png", dpi=180)
127 plt.close()
128
129
130 def main() -> None:
131     missing = [p for p in ["q1_result.csv", "q2_result.csv", "
        q3_result.csv", "q4_result.csv", "q5_result.csv"] if not (
            RESULTS / p).exists()]
132     if missing:
133         raise SystemExit(f"Missing_result_files: {missing}. Run
            run_q1.py...run_q5.py first.")
134     ours = collect_ours()
135     write_workbooks(ours)
136     write_figures(ours)
137     print("Wrote support_2025A/method_comparison.xlsx")
138     print("Wrote support_2025A/results_q1_q5.xlsx")
139     print("Wrote support_2025A/figures/*.png")
140
141
142 if __name__ == "__main__":
143     main()

```